

NAME : J Sai Harini

HT No : 2303A52426

Batch : 35

## ✓ Test Cases for Merge Sort

```
def binary_search(arr, target):
    """
    Performs a binary search to find the index of a target element in a sorted list.

    Args:
        arr (list): A sorted list of elements.
        target (any): The element to search for.

    Returns:
        int: The index of the target element if found, otherwise -1.

    Time Complexity:
        - Best Case: O(1) - When the target element is found at the middle of the list
          on the first comparison.
        - Average Case: O(log n) - On average, the search space is halved in each step
          until the element is found or the search space is empty.
        - Worst Case: O(log n) - When the target element is at one of the ends of the
          list or not present at all. In each step, the algorithm
          eliminates half of the remaining search space.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2 # Prevent potential overflow compared to (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

## ✓ Test Cases for Binary Search

```
# Test Case 1: Target found in the middle
my_list_1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
target_1 = 5
expected_1 = 4
result_1 = binary_search(my_list_1, target_1)
print(f"List: {my_list_1}, Target: {target_1}, Expected: {expected_1}, Result: {result_1}")
assert result_1 == expected_1, f"Test Case 1 Failed: Expected {expected_1}, got {result_1}"

# Test Case 2: Target found at the beginning
my_list_2 = [10, 20, 30, 40, 50]
target_2 = 10
expected_2 = 0
result_2 = binary_search(my_list_2, target_2)
print(f"List: {my_list_2}, Target: {target_2}, Expected: {expected_2}, Result: {result_2}")
assert result_2 == expected_2, f"Test Case 2 Failed: Expected {expected_2}, got {result_2}"

# Test Case 3: Target found at the end
my_list_3 = [10, 20, 30, 40, 50]
target_3 = 50
expected_3 = 4
result_3 = binary_search(my_list_3, target_3)
print(f"List: {my_list_3}, Target: {target_3}, Expected: {expected_3}, Result: {result_3}")
assert result_3 == expected_3, f"Test Case 3 Failed: Expected {expected_3}, got {result_3}"

# Test Case 4: Target not found
my_list_4 = [1, 3, 5, 7, 9]
```

```

target_4 = 4
expected_4 = -1
result_4 = binary_search(my_list_4, target_4)
print(f"List: {my_list_4}, Target: {target_4}, Expected: {expected_4}, Result: {result_4}")
assert result_4 == expected_4, f"Test Case 4 Failed: Expected {expected_4}, got {result_4}"

# Test Case 5: Empty list
my_list_5 = []
target_5 = 100
expected_5 = -1
result_5 = binary_search(my_list_5, target_5)
print(f"List: {my_list_5}, Target: {target_5}, Expected: {expected_5}, Result: {result_5}")
assert result_5 == expected_5, f"Test Case 5 Failed: Expected {expected_5}, got {result_5}"

# Test Case 6: List with one element, target found
my_list_6 = [7]
target_6 = 7
expected_6 = 0
result_6 = binary_search(my_list_6, target_6)
print(f"List: {my_list_6}, Target: {target_6}, Expected: {expected_6}, Result: {result_6}")
assert result_6 == expected_6, f"Test Case 6 Failed: Expected {expected_6}, got {result_6}"

# Test Case 7: List with one element, target not found
my_list_7 = [7]
target_7 = 8
expected_7 = -1
result_7 = binary_search(my_list_7, target_7)
print(f"List: {my_list_7}, Target: {target_7}, Expected: {expected_7}, Result: {result_7}")
assert result_7 == expected_7, f"Test Case 7 Failed: Expected {expected_7}, got {result_7}"

# Test Case 8: List with duplicate elements, target found
my_list_8 = [1, 2, 2, 3, 4, 4, 4, 5]
target_8 = 4
# Binary search might return any valid index for a duplicate target. We'll check if it's one of them.
result_8 = binary_search(my_list_8, target_8)
print(f"List: {my_list_8}, Target: {target_8}, Result: {result_8}")
assert result_8 in [4, 5, 6], f"Test Case 8 Failed: Target 4 not found at a valid index (4, 5, or 6), got {result_8}"

# Test Case 9: List with negative numbers
my_list_9 = [-5, -3, -1, 0, 2, 4]
target_9 = -1
expected_9 = 2
result_9 = binary_search(my_list_9, target_9)
print(f"List: {my_list_9}, Target: {target_9}, Expected: {expected_9}, Result: {result_9}")
assert result_9 == expected_9, f"Test Case 9 Failed: Expected {expected_9}, got {result_9}"

# Test Case 10: Larger list, target found
import random
large_list_sorted = sorted([random.randint(1, 1000) for _ in range(100)])
target_10 = large_list_sorted[random.randint(0, 99)] # Pick an existing element
result_10 = binary_search(large_list_sorted, target_10)
print(f"Large List (first 5): {large_list_sorted[:5]}..., Target: {target_10}, Result Index: {result_10}")
assert large_list_sorted[result_10] == target_10, f"Test Case 10 Failed: Element at {result_10} is not {target_10}"

# Test Case 11: Larger list, target not found
import random
large_list_sorted_2 = sorted([random.randint(1, 1000) for _ in range(100)])
target_11 = 9999 # A value very unlikely to be in the list
result_11 = binary_search(large_list_sorted_2, target_11)
print(f"Large List (first 5): {large_list_sorted_2[:5]}..., Target: {target_11}, Result: {result_11}")
assert result_11 == -1, f"Test Case 11 Failed: Expected -1, got {result_11}"

print("All binary_search test cases passed!")

```

```

List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Target: 5, Expected: 4, Result: 4
List: [10, 20, 30, 40, 50], Target: 10, Expected: 0, Result: 0
List: [10, 20, 30, 40, 50], Target: 50, Expected: 4, Result: 4
List: [1, 3, 5, 7, 9], Target: 4, Expected: -1, Result: -1
List: [], Target: 100, Expected: -1, Result: -1
List: [7], Target: 7, Expected: 0, Result: 0
List: [7], Target: 8, Expected: -1, Result: -1
List: [1, 2, 2, 3, 4, 4, 4, 5], Target: 4, Result: 5
List: [-5, -3, -1, 0, 2, 4], Target: -1, Expected: 2, Result: 2
Large List (first 5): [3, 16, 21, 24, 32]..., Target: 282, Result Index: 25
Large List (first 5): [11, 35, 37, 58, 93]..., Target: 9999, Result: -1
All binary_search test cases passed!

```

```

# Test Case 1: Empty list
empty_list = []
print(f"Original: {empty_list}, Sorted: {merge_sort(empty_list)}")
assert merge_sort(empty_list) == [], "Test Case 1 Failed: Empty list"

# Test Case 2: Single element list
single_element_list = [5]
print(f"Original: {single_element_list}, Sorted: {merge_sort(single_element_list)}")
assert merge_sort(single_element_list) == [5], "Test Case 2 Failed: Single element list"

# Test Case 3: Already sorted list
already_sorted_list = [1, 2, 3, 4, 5]
print(f"Original: {already_sorted_list}, Sorted: {merge_sort(already_sorted_list)}")
assert merge_sort(already_sorted_list) == [1, 2, 3, 4, 5], "Test Case 3 Failed: Already sorted list"

# Test Case 4: Reverse sorted list
reverse_sorted_list = [5, 4, 3, 2, 1]
print(f"Original: {reverse_sorted_list}, Sorted: {merge_sort(reverse_sorted_list)}")
assert merge_sort(reverse_sorted_list) == [1, 2, 3, 4, 5], "Test Case 4 Failed: Reverse sorted list"

# Test Case 5: List with duplicate elements
duplicate_list = [3, 1, 4, 1, 5, 9, 2, 6]
print(f"Original: {duplicate_list}, Sorted: {merge_sort(duplicate_list)}")
assert merge_sort(duplicate_list) == [1, 1, 2, 3, 4, 5, 6, 9], "Test Case 5 Failed: Duplicate elements"

# Test Case 6: List with negative numbers
negative_list = [-3, -1, -4, 0, -2]
print(f"Original: {negative_list}, Sorted: {merge_sort(negative_list)}")
assert merge_sort(negative_list) == [-4, -3, -2, -1, 0], "Test Case 6 Failed: Negative numbers"

# Test Case 7: Mixed list (positive, negative, zero)
mixed_list = [7, -2, 0, 4, -1, 3]
print(f"Original: {mixed_list}, Sorted: {merge_sort(mixed_list)}")
assert merge_sort(mixed_list) == [-2, -1, 0, 3, 4, 7], "Test Case 7 Failed: Mixed list"

# Test Case 8: Larger random list
import random
large_list = [random.randint(0, 1000) for _ in range(50)]
sorted_large_list = sorted(large_list)
print(f"Original (first 10): {large_list[:10]}..., Sorted (first 10): {merge_sort(large_list)[:10]}...")
assert merge_sort(large_list) == sorted_large_list, "Test Case 8 Failed: Larger random list"

print("All merge_sort test cases passed!")

```

```

Original: [], Sorted: []
Original: [5], Sorted: [5]
Original: [1, 2, 3, 4, 5], Sorted: [1, 2, 3, 4, 5]
Original: [5, 4, 3, 2, 1], Sorted: [1, 2, 3, 4, 5]
Original: [3, 1, 4, 1, 5, 9, 2, 6], Sorted: [1, 1, 2, 3, 4, 5, 6, 9]
Original: [-3, -1, -4, 0, -2], Sorted: [-4, -3, -2, -1, 0]
Original: [7, -2, 0, 4, -1, 3], Sorted: [-2, -1, 0, 3, 4, 7]
Original (first 10): [524, 220, 410, 455, 277, 598, 31, 718, 120, 421]..., Sorted (first 10): [31, 99, 120, 131, 145, 146, 159,
All merge_sort test cases passed!

```

## Task

Create an inventory management system in Python by defining a `Product` class (with ID, name, price, stock), implementing search functions (`search_by_id` using a dictionary for  $O(1)$  and `search_by_name` using binary search for  $O(\log n)$ ), implementing sort functions (`sort_by_price` and `sort_by_quantity` using Python's built-in `sorted()` for  $O(n \log n)$ ), and finally demonstrating and verifying all functionalities with sample data.

## Product Class Definition

### Subtask:

Define a `Product` class in Python to encapsulate product attributes such as ID, name, price, and stock quantity. This will serve as the basic data structure for inventory items.

**Reasoning:** The subtask requires defining a `Product` class with specified attributes and methods. I will create a Python code block to implement this class as instructed.

```
class Product:
    """
    Represents a product with ID, name, price, and stock quantity.
    """
    def __init__(self, product_id, name, price, stock):
        """
        Initializes a new Product instance.

        Args:
            product_id (int): Unique identifier for the product.
            name (str): Name of the product.
            price (float): Price of the product.
            stock (int): Current stock quantity of the product.
        """
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock

    def __repr__(self):
        """
        Returns a string representation of the Product object.
        """
        return f"Product(ID={self.product_id}, Name='{self.name}', Price={self.price}, Stock={self.stock})"

# Example usage (for verification, not part of the core task but good to include for testing)
# product1 = Product(1, 'Laptop', 1200.0, 10)
# print(product1)
# product2 = Product(2, 'Mouse', 25.0, 50)
# print(product2)
```

Start coding or [generate](#) with AI.

## ▼ Implement Search Functions

Subtask:

Implement Python functions for searching the inventory: `search_by_id(inventory_list, product_id)` and `search_by_name(inventory_list, product_name)`.

## ▼ Implementation Plan for Search Functions

To implement the search functionalities as required, we will define two distinct Python functions:

1. `search_by_id(inventory_list, product_id)`:
  - **Data Structure:** We will first convert the `inventory_list` (which is a list of `Product` objects) into a dictionary. The keys of this dictionary will be the `product_id`s, and the values will be the corresponding `Product` objects. This allows for direct,  $O(1)$  access by ID.
  - **Lookup:** Once the dictionary is constructed, we can use a simple dictionary lookup to retrieve the `Product` object associated with the given `product_id`. If the `product_id` is not found as a key, we return `None`.
2. `search_by_name(inventory_list, product_name)`:
  - **Preparation:** For efficient searching by name, the list must be sorted by name. We will create a *copy* of the `inventory_list` to ensure the original list's order is preserved. This copied list will then be sorted using Python's `sorted()` function, with a `lambda` expression to specify that sorting should be based on the `name` attribute of each `Product` object.
  - **Binary Search:** We will adapt the existing `binary_search` logic (from cell 0752b333) to work with `Product` objects. Instead of comparing simple integer values, the binary search will compare the `name` attribute of the `Product` objects in the sorted list with the `product_name` target. The binary search will iteratively narrow down the search space until the product is found or the search space is exhausted. If found, the `Product` object is returned; otherwise, `None`.

**Reasoning:** Now I will implement the `search_by_id` and `search_by_name` functions based on the detailed plan provided in the previous markdown block. This code block will contain the Python definitions for both functions.

```
def search_by_id(inventory_list, product_id):
    """
    Searches for a Product object by its ID in the inventory_list.

    Args:
        inventory_list (list): A list of Product objects.
        product_id (int): The ID of the product to search for.

    Returns:
        Product: The Product object if found, otherwise None.
    Time Complexity: O(n) in worst case (to build dictionary), O(1) for subsequent lookups after dictionary is built.
    If the dictionary is built once and reused, then it's O(1).
    """
    # Create a dictionary for O(1) lookup by product_id
    id_to_product = {product.product_id: product for product in inventory_list}
    return id_to_product.get(product_id)

def search_by_name(inventory_list, product_name):
    """
    Searches for a Product object by its name in the inventory_list using binary search.

    Args:
        inventory_list (list): A list of Product objects.
        product_name (str): The name of the product to search for.

    Returns:
        Product: The Product object if found, otherwise None.
    Time Complexity: O(N log N) due to sorting, O(log N) for binary search on sorted list.
    """
    # Create a copy and sort it by product name
    sorted_inventory = sorted(inventory_list, key=lambda product: product.name)

    left, right = 0, len(sorted_inventory) - 1

    while left <= right:
        mid = left + (right - left) // 2
        current_product_name = sorted_inventory[mid].name

        if current_product_name == product_name:
            return sorted_inventory[mid]
        elif current_product_name < product_name:
            left = mid + 1
        else:
            right = mid - 1

    return None

print("Search functions `search_by_id` and `search_by_name` defined.")
```

Search functions `search\_by\_id` and `search\_by\_name` defined.

## Task

### Implement Sort Functions

#### Subtask:

Implement Python functions for sorting product records: `sort_by_price(inventory_list)` and `sort_by_stock(inventory_list)`. These functions will utilize Python's built-in `sorted()` function (Timsort), providing efficient  $O(n \log n)$  performance.

## Implement Sort Functions

#### Subtask:

Implement Python functions for sorting product records: `sort_by_price(inventory_list)` and `sort_by_stock(inventory_list)`. These functions will utilize Python's built-in `sorted()` function (Timsort), providing efficient  $O(n \log n)$  performance.

## Implementation Plan for Sort Functions

To implement the sorting functionalities, we will define two Python functions:

### 1. `sort_by_price(inventory_list)`:

- **Sorting Logic:** This function will take an `inventory_list` (a list of `Product` objects) and sort it based on the `price` attribute of each product.
- **Method:** We will use Python's built-in `sorted()` function, which provides efficient  $O(N \log N)$  time complexity. A `lambda` expression will be used as the `key` argument to `sorted()` to specify that sorting should be performed on the `price` attribute (e.g., `key=lambda product: product.price`).
- **Return Value:** The function will return a *new* list containing the `Product` objects sorted by price, leaving the original `inventory_list` unchanged.

### 2. `sort_by_stock(inventory_list)`:

- **Sorting Logic:** Similar to `sort_by_price`, this function will sort the `inventory_list` based on the `stock` attribute of each `Product` object.
- **Method:** Again, Python's `sorted()` function will be used, with a `lambda` expression targeting the `stock` attribute (e.g., `key=lambda product: product.stock`).
- **Return Value:** The function will return a *new* list containing the `Product` objects sorted by stock quantity, without modifying the original `inventory_list`.

**Reasoning:** I will now implement the `sort_by_price` and `sort_by_stock` functions in a Python code block, following the detailed plan previously outlined. This includes using `sorted()` with lambda functions and adding appropriate docstrings.

```
def sort_by_price(inventory_list):
    """
    Sorts a list of Product objects by their price.

    Args:
        inventory_list (list): A list of Product objects.

    Returns:
        list: A new list containing Product objects sorted by price.
    Time Complexity: O(N log N) due to Python's built-in sorted() (Timsort).
    """
    return sorted(inventory_list, key=lambda product: product.price)

def sort_by_stock(inventory_list):
    """
    Sorts a list of Product objects by their stock quantity.

    Args:
        inventory_list (list): A list of Product objects.

    Returns:
        list: A new list containing Product objects sorted by stock quantity.
    Time Complexity: O(N log N) due to Python's built-in sorted() (Timsort).
    """
    return sorted(inventory_list, key=lambda product: product.stock)

print("Sort functions `sort_by_price` and `sort_by_stock` defined.")

Sort functions `sort_by_price` and `sort_by_stock` defined.
```

## Demonstrate and Verify Algorithms

Subtask:

Create sample data and verify the functionality of the implemented `Product` class, search functions (`search_by_id`, `search_by_name`), and sort functions (`sort_by_price`, `sort_by_stock`).

**Reasoning:** I will create sample `Product` data and then demonstrate and verify the functionality of the `Product` class, `search_by_id`, `search_by_name`, `sort_by_price`, and `sort_by_stock` functions as per the instructions in the subtask.

```

print("\n--- Demonstrating Product Class and Functions ---\n")

# 1. Create a sample list of Product objects
products = [
    Product(101, 'Laptop Pro', 1200.00, 15),
    Product(102, 'Gaming Mouse', 45.99, 120),
    Product(103, 'Mechanical Keyboard', 89.99, 50),
    Product(104, 'Monitor 27 inch', 299.99, 30),
    Product(105, 'Webcam HD', 65.00, 80),
    Product(106, 'USB Hub', 12.50, 200),
    Product(107, 'Laptop Stand', 25.00, 75)
]
print("Sample Inventory Created:")
for p in products:
    print(p)

print("\n--- Testing Search Functions ---\n")

# 2a. Demonstrate search_by_id for an existing product
search_id_exists = 103
found_product_id = search_by_id(products, search_id_exists)
print(f"Searching by ID '{search_id_exists}': {found_product_id}")
assert found_product_id is not None and found_product_id.product_id == search_id_exists, \
    f"Test Failed: search_by_id for existing ID {search_id_exists}"

# 2b. Demonstrate search_by_id for a non-existing product
search_id_non_exists = 999
found_product_id_non = search_by_id(products, search_id_non_exists)
print(f"Searching by ID '{search_id_non_exists}': {found_product_id_non}")
assert found_product_id_non is None, \
    f"Test Failed: search_by_id for non-existing ID {search_id_non_exists}"

# 3a. Demonstrate search_by_name for an existing product
search_name_exists = 'Webcam HD'
found_product_name = search_by_name(products, search_name_exists)
print(f"Searching by Name '{search_name_exists}': {found_product_name}")
assert found_product_name is not None and found_product_name.name == search_name_exists, \
    f"Test Failed: search_by_name for existing Name '{search_name_exists}'"

# 3b. Demonstrate search_by_name for a non-existing product
search_name_non_exists = 'Smart Watch'
found_product_name_non = search_by_name(products, search_name_non_exists)
print(f"Searching by Name '{search_name_non_exists}': {found_product_name_non}")
assert found_product_name_non is None, \
    f"Test Failed: search_by_name for non-existing Name '{search_name_non_exists}'"

print("Search functions verified successfully.")

print("\n--- Testing Sort Functions ---\n")

# 4. Demonstrate sort_by_price function
sorted_by_price_products = sort_by_price(products)
print("Products Sorted by Price:")
for p in sorted_by_price_products:
    print(p)
# Verification (manual check of first and last element, or a simple loop)
assert sorted_by_price_products[0].price <= sorted_by_price_products[1].price, \
    "Test Failed: sort_by_price did not sort correctly"
assert sorted_by_price_products[-1].price >= sorted_by_price_products[-2].price, \
    "Test Failed: sort_by_price did not sort correctly"

# 5. Demonstrate sort_by_stock function
sorted_by_stock_products = sort_by_stock(products)
print("\nProducts Sorted by Stock Quantity:")
for p in sorted_by_stock_products:
    print(p)
# Verification (manual check of first and last element, or a simple loop)
assert sorted_by_stock_products[0].stock <= sorted_by_stock_products[1].stock, \
    "Test Failed: sort_by_stock did not sort correctly"
assert sorted_by_stock_products[-1].stock >= sorted_by_stock_products[-2].stock, \
    "Test Failed: sort_by_stock did not sort correctly"

print("Sort functions verified successfully.")

print("\nAll demonstrations and verifications completed successfully.")

```

```

--- Demonstrating Product Class and Functions ---

Sample Inventory Created:
Product(ID=101, Name='Laptop Pro', Price=1200.0, Stock=15)
Product(ID=102, Name='Gaming Mouse', Price=45.99, Stock=120)
Product(ID=103, Name='Mechanical Keyboard', Price=89.99, Stock=50)
Product(ID=104, Name='Monitor 27 inch', Price=299.99, Stock=30)
Product(ID=105, Name='Webcam HD', Price=65.0, Stock=80)
Product(ID=106, Name='USB Hub', Price=12.5, Stock=200)
Product(ID=107, Name='Laptop Stand', Price=25.0, Stock=75)

--- Testing Search Functions ---

Searching by ID '103': Product(ID=103, Name='Mechanical Keyboard', Price=89.99, Stock=50)
Searching by ID '999': None
Searching by Name 'Webcam HD': Product(ID=105, Name='Webcam HD', Price=65.0, Stock=80)
Searching by Name 'Smart Watch': None
Search functions verified successfully.

--- Testing Sort Functions ---

Products Sorted by Price:
Product(ID=106, Name='USB Hub', Price=12.5, Stock=200)
Product(ID=107, Name='Laptop Stand', Price=25.0, Stock=75)
Product(ID=102, Name='Gaming Mouse', Price=45.99, Stock=120)
Product(ID=105, Name='Webcam HD', Price=65.0, Stock=80)
Product(ID=103, Name='Mechanical Keyboard', Price=89.99, Stock=50)
Product(ID=104, Name='Monitor 27 inch', Price=299.99, Stock=30)
Product(ID=101, Name='Laptop Pro', Price=1200.0, Stock=15)

Products Sorted by Stock Quantity:
Product(ID=101, Name='Laptop Pro', Price=1200.0, Stock=15)
Product(ID=104, Name='Monitor 27 inch', Price=299.99, Stock=30)
Product(ID=103, Name='Mechanical Keyboard', Price=89.99, Stock=50)
Product(ID=107, Name='Laptop Stand', Price=25.0, Stock=75)
Product(ID=105, Name='Webcam HD', Price=65.0, Stock=80)
Product(ID=102, Name='Gaming Mouse', Price=45.99, Stock=120)
Product(ID=106, Name='USB Hub', Price=12.5, Stock=200)
Sort functions verified successfully.

All demonstrations and verifications completed successfully.

```

## Final Task

### Subtask:

Summarize the Inventory Management System solution, highlighting the chosen algorithms and their benefits for managing product records efficiently.

### Summary:

#### Data Analysis Key Findings

- **Sort Function Implementation:** Two Python functions, `sort_by_price` and `sort_by_stock`, were successfully implemented. These functions leverage Python's built-in `sorted()` function, which uses Timsort, ensuring an efficient  $O(N \log N)$  time complexity for sorting product records by their respective attributes (`price` and `stock`).
- **Sample Data Creation:** A list of seven `Product` objects, each with unique IDs, names, prices, and stock quantities, was successfully created to serve as sample inventory data for verification.
- **Search Function Verification:**
  - `search_by_id`: Successfully retrieved a product with an existing ID (e.g., ID 103, 'Mechanical Keyboard') and correctly returned `None` for a non-existing ID (999).
  - `search_by_name`: Successfully retrieved a product with an existing name (e.g., 'Webcam HD') and correctly returned `None` for a non-existing name ('Smart Watch').
- **Sort Function Verification:**
  - `sort_by_price`: Successfully sorted the sample products in ascending order by price, confirming products like 'USB Hub' (\$12.50) appeared first and 'Laptop Pro' (\$1200.00) appeared last in the sorted list.
  - `sort_by_stock`: Successfully sorted the sample products in ascending order by stock quantity, confirming products like 'Laptop Pro' (15 units) appeared first and 'USB Hub' (200 units) appeared last.



- **Verification Success:** All embedded assertions for both search and sort functionalities passed, confirming the correct behavior and implementation of all tested components.

## Insights or Next Steps

- The implementation of efficient  $O(N \log N)$  sorting functions is crucial for managing a dynamic inventory, allowing quick retrieval of products based on price or stock levels, which is beneficial for pricing strategies or stock reordering.
- Consider extending the sorting functionality to include options for descending order, secondary sort keys (e.g., sort by price, then by name for ties), or more complex filtering criteria to enhance inventory analysis capabilities.

Start coding or [generate](#) with AI.

## Task

Develop an online food delivery platform system in Python. This system should include:

1. **Algorithm Recommendations and Justification:** Provide a detailed explanation of chosen search and sort algorithms (e.g.,  $O(1)$ ,  $O(\log n)$ ,  $O(n \log n)$ ) for operations like searching by order ID and sorting by delivery time or price, including a markdown table.
2. **Order Class Definition:** Define an `Order` class with attributes such as order ID, restaurant name, delivery time, price, and order status.
3. **Implement Search Function:** Create a Python function `search_by_order_id(order_list, order_id)` for efficient (average  $O(1)$ ) searching using a dictionary.
4. **Implement Sort Functions:** Create Python functions `sort_by_delivery_time(order_list)` and `sort_by_price(order_list)` utilizing Python's built-in `sorted()` for  $O(n \log n)$  performance.
5. **Demonstrate and Verify Algorithms:** Create a sample list of `Order` objects and demonstrate the usage of the implemented search and sort functions, verifying correctness with examples of existing/non-existing orders and sorted lists.
6. **Summarize:** Provide a summary of the solution, highlighting the chosen algorithms and their benefits.

## Algorithm Recommendations and Justification

### Subtask:

Provide a detailed explanation of the chosen search and sort algorithms for the online food delivery platform, including a markdown table that maps each operation to its recommended algorithm and a justification based on efficiency and suitability (e.g.,  $O(1)$ ,  $O(\log n)$ ,  $O(n \log n)$ ). This will cover searching by order ID and sorting by delivery time or price.

## Algorithm Recommendations and Justification

For an online food delivery platform, efficient management of orders is crucial. This involves quickly finding specific orders and organizing them based on various criteria. Below, we detail the recommended algorithms for common operations like searching by order ID and sorting by delivery time or price, along with their justifications.

### Chosen Algorithms and Their Benefits

#### 1. Searching by Order ID ( $O(1)$ Average Time Complexity)

- **Recommended Algorithm:** Hash Map (Dictionary in Python)
- **Justification:** Order IDs are unique identifiers. Using a hash map (dictionary in Python) allows for direct, almost instantaneous retrieval of an order given its ID. The average time complexity for lookup in a hash map is  $O(1)$ , making it exceptionally fast for accessing individual order records. This is critical for customer service inquiries, order tracking, and internal order management, where immediate access to specific orders is paramount.

#### 2. Sorting by Delivery Time ( $O(N \log N)$ Time Complexity)

- **Recommended Algorithm:** Timsort (Python's built-in `sorted()` function)
- **Justification:** Delivery time is a frequently used criterion for organizing orders, both for dispatch and customer display. Timsort, which is a hybrid sorting algorithm combining merge sort and insertion sort, offers an optimal average and worst-case time complexity of  $O(N \log N)$ . It is efficient for a wide range of real-world data, including partially sorted data, making it suitable for dynamically updated order lists. Its stability also ensures that orders with the same delivery time maintain their relative original order, which can be beneficial for fairness or secondary sorting criteria.

3. Sorting by Price (O(N log N) Time Complexity)

- **Recommended Algorithm:** Timsort (Python's built-in `sorted()` function)
- **Justification:** Sorting orders or menu items by price (ascending or descending) is essential for customer experience, filtering, and promotional activities. Similar to sorting by delivery time, Timsort's O(N log N) efficiency makes it ideal for handling datasets of varying sizes. Its stability and performance characteristics ensure that price-based sorting is both fast and reliable, providing a smooth user experience when browsing or analyzing pricing tiers.

Summary Table of Algorithm Recommendations

| Operation             | Recommended Algorithm | Time Complexity | Justification                                                                                        |
|-----------------------|-----------------------|-----------------|------------------------------------------------------------------------------------------------------|
| Search by Order ID    | Hash Map (Dictionary) | O(1) (average)  | Provides direct and near-instantaneous access, crucial for unique identifiers and rapid retrieval.   |
| Sort by Delivery Time | Timsort               | O(N log N)      | Efficient and stable general-purpose sort, suitable for various data states, critical for logistics. |
| Sort by Price         | Timsort               | O(N log N)      | Efficient and stable, good for user display, filtering, and analytical purposes.                     |

Order Class Definition

Subtask:

Define an `Order` class in Python to encapsulate order attributes such as order ID, restaurant name, delivery time, price, and order status. This will serve as the basic data structure for order records.

**Reasoning:** The subtask requires defining an `Order` class with specified attributes and methods. I will create a Python code block to implement this class as instructed, including the `__init__` and `__repr__` methods.

```
from datetime import datetime

class Order:
    """
    Represents an order with ID, restaurant name, delivery time, price, and status.
    """
    def __init__(self, order_id, restaurant_name, delivery_time, price, order_status):
        """
        Initializes a new Order instance.

        Args:
            order_id (int): Unique identifier for the order.
            restaurant_name (str): Name of the restaurant.
            delivery_time (datetime or str): Expected or actual delivery time.
            price (float): Total price of the order.
            order_status (str): Current status of the order (e.g., 'Pending', 'Delivered').
        """
        self.order_id = order_id
        self.restaurant_name = restaurant_name
        self.delivery_time = delivery_time
        self.price = price
        self.order_status = order_status

    def __repr__(self):
        """
        Returns a string representation of the Order object.
        """
        return (
            f"Order(ID={self.order_id}, Restaurant='{self.restaurant_name}', "
            f"DeliveryTime='{self.delivery_time}', Price={self.price:.2f}, "
            f"Status='{self.order_status}')"
        )

print("Order class defined.")
```

Order class defined.

**Reasoning:** The next step is to implement the `search_by_order_id` function as described in the subtask and the algorithm recommendations. This function will use a dictionary for O(1) average time complexity.

```
def search_by_order_id(order_list, order_id):
    """
    Searches for an Order object by its ID in the order_list.
```

```

Args:
    order_list (list): A list of Order objects.
    order_id (int): The ID of the order to search for.

Returns:
    Order: The Order object if found, otherwise None.
Time Complexity: O(n) in worst case (to build dictionary), O(1) for subsequent lookups after dictionary is built.
If the dictionary is built once and reused, then it's O(1).
"""
# Create a dictionary for O(1) lookup by order_id
id_to_order = {order.order_id: order for order in order_list}
return id_to_order.get(order_id)

print("Search function `search_by_order_id` defined.")

```

Search function `search\_by\_order\_id` defined.

## Implement Sort Functions

### Subtask:

Implement Python functions for sorting order records: `sort_by_delivery_time(order_list)` and `sort_by_price(order_list)`. These functions will utilize Python's built-in `sorted()` for  $O(n \log n)$  performance.

## Implement Sort Functions

### Subtask:

Implement Python functions for sorting order records: `sort_by_delivery_time(order_list)` and `sort_by_price(order_list)`. These functions will utilize Python's built-in `sorted()` for  $O(n \log n)$  performance.

## Implementation Plan for Sort Functions

To implement the sorting functionalities, we will define two Python functions:

### 1. `sort_by_delivery_time(order_list)`:

- **Sorting Logic:** This function will take an `order_list` (a list of `Order` objects) and sort it based on the `delivery_time` attribute of each order.
- **Method:** We will use Python's built-in `sorted()` function, which provides efficient  $O(N \log N)$  time complexity. A `lambda` expression will be used as the `key` argument to `sorted()` to specify that sorting should be performed on the `delivery_time` attribute (e.g., `key=lambda order: order.delivery_time`).
- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by delivery time, leaving the original `order_list` unchanged.

### 2. `sort_by_price(order_list)`:

- **Sorting Logic:** Similar to `sort_by_delivery_time`, this function will sort the `order_list` based on the `price` attribute of each `Order` object.
- **Method:** Again, Python's `sorted()` function will be used, with a `lambda` expression targeting the `price` attribute (e.g., `key=lambda order: order.price`).
- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by price, without modifying the original `order_list`.

## Implementation Plan for Sort Functions

To implement the sorting functionalities, we will define two Python functions:

### 1. `sort_by_delivery_time(order_list)`:

- **Sorting Logic:** This function will take an `order_list` (a list of `Order` objects) and sort it based on the `delivery_time` attribute of each order.
- **Method:** We will use Python's built-in `sorted()` function, which provides efficient  $O(N \log N)$  time complexity. A `lambda` expression will be used as the `key` argument to `sorted()` to specify that sorting should be performed on the

`delivery_time` attribute (e.g., `key=lambda order: order.delivery_time`).

- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by delivery time, leaving the original `order_list` unchanged.

## 2. `sort_by_price(order_list)`:

- **Sorting Logic:** Similar to `sort_by_delivery_time`, this function will sort the `order_list` based on the `price` attribute of each `Order` object.
- **Method:** Again, Python's `sorted()` function will be used, with a `lambda` expression targeting the `price` attribute (e.g., `key=lambda order: order.price`).
- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by price, without modifying the original `order_list`.

## ✓ Implementation Plan for Sort Functions

To implement the sorting functionalities, we will define two Python functions:

### 1. `sort_by_delivery_time(order_list)`:

- **Sorting Logic:** This function will take an `order_list` (a list of `Order` objects) and sort it based on the `delivery_time` attribute of each order.
- **Method:** We will use Python's built-in `sorted()` function, which provides efficient  $O(N \log N)$  time complexity. A `lambda` expression will be used as the `key` argument to `sorted()` to specify that sorting should be performed on the `delivery_time` attribute (e.g., `key=lambda order: order.delivery_time`).
- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by delivery time, leaving the original `order_list` unchanged.

### 2. `sort_by_price(order_list)`:

- **Sorting Logic:** Similar to `sort_by_delivery_time`, this function will sort the `order_list` based on the `price` attribute of each `Order` object.
- **Method:** Again, Python's `sorted()` function will be used, with a `lambda` expression targeting the `price` attribute (e.g., `key=lambda order: order.price`).
- **Return Value:** The function will return a *new* list containing the `Order` objects sorted by price, without modifying the original `order_list`.

**Reasoning:** The detailed implementation plan for sort functions has been provided. Now, I will implement the `sort_by_delivery_time` and `sort_by_price` functions in a Python code block, following the outlined plan.

```
def sort_by_delivery_time(order_list):
    """
    Sorts a list of Order objects by their delivery time.

    Args:
        order_list (list): A list of Order objects.

    Returns:
        list: A new list containing Order objects sorted by delivery time.
        Time Complexity: O(N log N) due to Python's built-in sorted() (Timsort).
    """
    return sorted(order_list, key=lambda order: order.delivery_time)

def sort_by_price(order_list):
    """
    Sorts a list of Order objects by their price.

    Args:
        order_list (list): A list of Order objects.

    Returns:
        list: A new list containing Order objects sorted by price.
        Time Complexity: O(N log N) due to Python's built-in sorted() (Timsort).
    """
    return sorted(order_list, key=lambda order: order.price)

print("Sort functions `sort_by_delivery_time` and `sort_by_price` defined.")
```

```
Sort functions `sort_by_delivery_time` and `sort_by_price` defined.
```

## Demonstrate and Verify Algorithms

### Subtask:

Create a sample list of `Order` objects and demonstrate the usage of the implemented search and sort functions, verifying correctness with examples of existing/non-existing orders and sorted lists.

## ▼ Demonstrate and Verify Algorithms

### Subtask:

Create a sample list of `Order` objects and demonstrate the usage of the implemented search and sort functions, verifying correctness with examples of existing/non-existing orders and sorted lists.

## Implementation Plan for Demonstration and Verification

To demonstrate and verify the functionality of the `Order` class, search functions (`search_by_order_id`), and sort functions (`sort_by_delivery_time`, `sort_by_price`), we will follow these steps:

1. **Create Sample `Order` Data:** Generate a list of several `Order` objects with diverse `order_id`, `restaurant_name`, `delivery_time`, `price`, and `order_status` values. Ensure there's a mix of different delivery times and prices for effective sorting demonstrations.
  - `delivery_time` will be `datetime` objects to ensure correct sorting.
2. **Demonstrate `Order` Class:** Print the created sample `Order` objects to show their `__repr__` method works as expected.
3. **Verify `search_by_order_id` Function:**
  - **Existing Order:** Search for an `order_id` that is known to exist in the sample list and assert that the correct `Order` object is returned.
  - **Non-Existing Order:** Search for an `order_id` that does not exist in the sample list and assert that `None` is returned.
4. **Verify `sort_by_delivery_time` Function:**
  - Call `sort_by_delivery_time` with the sample `order_list`.
  - Print the sorted list.
  - Assert that the resulting list is indeed sorted by `delivery_time` (e.g., by checking consecutive elements or comparing with a manually sorted list).
5. **Verify `sort_by_price` Function:**
  - Call `sort_by_price` with the sample `order_list`.
  - Print the sorted list.
  - Assert that the resulting list is indeed sorted by `price` (e.g., by checking consecutive elements or comparing with a manually sorted list).

This comprehensive plan will ensure all required functionalities are thoroughly tested and demonstrated.

## Implementation Plan for Demonstration and Verification

To demonstrate and verify the functionality of the `Order` class, search functions (`search_by_order_id`), and sort functions (`sort_by_delivery_time`, `sort_by_price`), we will follow these steps:

1. **Create Sample `Order` Data:** Generate a list of several `Order` objects with diverse `order_id`, `restaurant_name`, `delivery_time`, `price`, and `order_status` values. Ensure there's a mix of different delivery times and prices for effective sorting demonstrations.
  - `delivery_time` will be `datetime` objects to ensure correct sorting.
2. **Demonstrate `Order` Class:** Print the created sample `Order` objects to show their `__repr__` method works as expected.
3. **Verify `search_by_order_id` Function:**

- **Existing Order:** Search for an `order_id` that is known to exist in the sample list and assert that the correct `Order` object is returned.
- **Non-Existing Order:** Search for an `order_id` that does not exist in the sample list and assert that `None` is returned.

#### 4. Verify `sort_by_delivery_time` Function:

- Call `sort_by_delivery_time` with the sample `order_list`.
- Print the sorted list.
- Assert that the resulting list is indeed sorted by `delivery_time` (e.g., by checking consecutive elements or comparing with a manually sorted list).

#### 5. Verify `sort_by_price` Function:

- Call `sort_by_price` with the sample `order_list`.
- Print the sorted list.
- Assert that the resulting list is indeed sorted by `price` (e.g., by checking consecutive elements or comparing with a manually sorted list).

This comprehensive plan will ensure all required functionalities are thoroughly tested and demonstrated.

## Implementation Plan for Demonstration and Verification

To demonstrate and verify the functionality of the `Order` class, search functions (`search_by_order_id`), and sort functions (`sort_by_delivery_time`, `sort_by_price`), we will follow these steps:

- Create Sample `Order` Data:** Generate a list of several `Order` objects with diverse `order_id`, `restaurant_name`, `delivery_time`, `price`, and `order_status` values. Ensure there's a mix of different delivery times and prices for effective sorting demonstrations.
  - `delivery_time` will be `datetime` objects to ensure correct sorting.
- Demonstrate `Order` Class:** Print the created sample `Order` objects to show their `__repr__` method works as expected.
- Verify `search_by_order_id` Function:**
  - **Existing Order:** Search for an `order_id` that is known to exist in the sample list and assert that the correct `Order` object is returned.
  - **Non-Existing Order:** Search for an `order_id` that does not exist in the sample list and assert that `None` is returned.
- Verify `sort_by_delivery_time` Function:**
  - Call `sort_by_delivery_time` with the sample `order_list`.
  - Print the sorted list.
  - Assert that the resulting list is indeed sorted by `delivery_time` (e.g., by checking consecutive elements or comparing with a manually sorted list).
- Verify `sort_by_price` Function:**
  - Call `sort_by_price` with the sample `order_list`.
  - Print the sorted list.
  - Assert that the resulting list is indeed sorted by `price` (e.g., by checking consecutive elements or comparing with a manually sorted list).

This comprehensive plan will ensure all required functionalities are thoroughly tested and demonstrated.

## Implementation Plan for Demonstration and Verification

To demonstrate and verify the functionality of the `Order` class, search functions (`search_by_order_id`), and sort functions (`sort_by_delivery_time`, `sort_by_price`), we will follow these steps:

- Create Sample `Order` Data:** Generate a list of several `Order` objects with diverse `order_id`, `restaurant_name`, `delivery_time`, `price`, and `order_status` values. Ensure there's a mix of different delivery times and prices for effective sorting demonstrations.
  - `delivery_time` will be `datetime` objects to ensure correct sorting.
- Demonstrate `Order` Class:** Print the created sample `Order` objects to show their `__repr__` method works as expected.
- Verify `search_by_order_id` Function:**

- **Existing Order:** Search for an `order_id` that is known to exist in the sample list and assert that the correct `Order` object is returned.
- **Non-Existing Order:** Search for an `order_id` that does not exist in the sample list and assert that `None` is returned.

#### 4. Verify `sort_by_delivery_time` Function:

- Call `sort_by_delivery_time` with the sample `order_list`.
- Print the sorted list.
- Assert that the resulting list is indeed sorted by `delivery_time` (e.g., by checking consecutive elements or comparing with a manually sorted list).

#### 5. Verify `sort_by_price` Function:

- Call `sort_by_price` with the sample `order_list`.
- Print the sorted list.
- Assert that the resulting list is indeed sorted by `price` (e.g., by checking consecutive elements or comparing with a manually sorted list).

This comprehensive plan will ensure all required functionalities are thoroughly tested and demonstrated.

**Reasoning:** The implementation plan for demonstration and verification has been outlined. Now, I will implement the code to create sample data, and then demonstrate and verify the functionality of the `Order` class, search, and sort functions as per the detailed plan.

```
from datetime import datetime, timedelta

print("\n--- Demonstrating Order Class and Functions ---\n")

# 1. Create a sample list of Order objects
# Using datetime objects for delivery_time for accurate sorting
orders = [
    Order(1, 'Pizza Palace', datetime.now() + timedelta(minutes=30), 25.50, 'Pending'),
    Order(2, 'Burger Joint', datetime.now() + timedelta(minutes=45), 15.00, 'Preparing'),
    Order(3, 'Sushi Spot', datetime.now() + timedelta(minutes=20), 40.75, 'Delivered'),
    Order(4, 'Taco Truck', datetime.now() + timedelta(minutes=30), 12.25, 'Pending'), # Same delivery time as order 1
    Order(5, 'Pasta House', datetime.now() + timedelta(minutes=60), 32.00, 'Out for Delivery')
]
print("Sample Order List Created:")
for o in orders:
    print(o)

print("\n--- Testing Search Function (search_by_order_id) ---\n")

# 2a. Demonstrate search_by_order_id for an existing order
search_id_exists = 3
found_order_id = search_by_order_id(orders, search_id_exists)
print(f"Searching for Order ID '{search_id_exists}': {found_order_id}")
assert found_order_id is not None and found_order_id.order_id == search_id_exists, \
    f"Test Failed: search_by_order_id for existing ID {search_id_exists}"

# 2b. Demonstrate search_by_order_id for a non-existing order
search_id_non_exists = 99
found_order_id_non = search_by_order_id(orders, search_id_non_exists)
print(f"Searching for Order ID '{search_id_non_exists}': {found_order_id_non}")
assert found_order_id_non is None, \
    f"Test Failed: search_by_order_id for non-existing ID {search_id_non_exists}"

print("Search function verified successfully.")

print("\n--- Testing Sort Functions ---\n")

# 3. Demonstrate sort_by_delivery_time function
sorted_by_time_orders = sort_by_delivery_time(orders)
print("Orders Sorted by Delivery Time:")
for o in sorted_by_time_orders:
    print(o)
# Verification: check if sorted correctly (asc)
for i in range(len(sorted_by_time_orders) - 1):
    assert sorted_by_time_orders[i].delivery_time <= sorted_by_time_orders[i+1].delivery_time, \
        f"Test Failed: sort_by_delivery_time not sorted correctly at index {i}"

# 4. Demonstrate sort_by_price function
sorted_by_price_orders = sort_by_price(orders)
print("\nOrders Sorted by Price:")
```



```

for o in sorted_by_price_orders:
    print(o)
# Verification: check if sorted correctly (asc)
for i in range(len(sorted_by_price_orders) - 1):
    assert sorted_by_price_orders[i].price <= sorted_by_price_orders[i+1].price, \
        f"Test Failed: sort_by_price not sorted correctly at index {i}"

print("Sort functions verified successfully.")

print("\nAll demonstrations and verifications completed successfully.")

--- Demonstrating Order Class and Functions ---

Sample Order List Created:
Order(ID=1, Restaurant='Pizza Palace', DeliveryTime='2026-02-23 09:07:38.880328', Price=25.50, Status='Pending')
Order(ID=2, Restaurant='Burger Joint', DeliveryTime='2026-02-23 09:22:38.880343', Price=15.00, Status='Preparing')
Order(ID=3, Restaurant='Sushi Spot', DeliveryTime='2026-02-23 08:57:38.880346', Price=40.75, Status='Delivered')
Order(ID=4, Restaurant='Taco Truck', DeliveryTime='2026-02-23 09:07:38.880349', Price=12.25, Status='Pending')
Order(ID=5, Restaurant='Pasta House', DeliveryTime='2026-02-23 09:37:38.880350', Price=32.00, Status='Out for Delivery')

--- Testing Search Function (search_by_order_id) ---

Searching for Order ID '3': Order(ID=3, Restaurant='Sushi Spot', DeliveryTime='2026-02-23 08:57:38.880346', Price=40.75, Status=
Searching for Order ID '99': None
Search function verified successfully.

--- Testing Sort Functions ---

Orders Sorted by Delivery Time:
Order(ID=3, Restaurant='Sushi Spot', DeliveryTime='2026-02-23 08:57:38.880346', Price=40.75, Status='Delivered')
Order(ID=1, Restaurant='Pizza Palace', DeliveryTime='2026-02-23 09:07:38.880328', Price=25.50, Status='Pending')
Order(ID=4, Restaurant='Taco Truck', DeliveryTime='2026-02-23 09:07:38.880349', Price=12.25, Status='Pending')
Order(ID=2, Restaurant='Burger Joint', DeliveryTime='2026-02-23 09:22:38.880343', Price=15.00, Status='Preparing')
Order(ID=5, Restaurant='Pasta House', DeliveryTime='2026-02-23 09:37:38.880350', Price=32.00, Status='Out for Delivery')

Orders Sorted by Price:
Order(ID=4, Restaurant='Taco Truck', DeliveryTime='2026-02-23 09:07:38.880349', Price=12.25, Status='Pending')
Order(ID=2, Restaurant='Burger Joint', DeliveryTime='2026-02-23 09:22:38.880343', Price=15.00, Status='Preparing')
Order(ID=1, Restaurant='Pizza Palace', DeliveryTime='2026-02-23 09:07:38.880328', Price=25.50, Status='Pending')
Order(ID=5, Restaurant='Pasta House', DeliveryTime='2026-02-23 09:37:38.880350', Price=32.00, Status='Out for Delivery')
Order(ID=3, Restaurant='Sushi Spot', DeliveryTime='2026-02-23 08:57:38.880346', Price=40.75, Status='Delivered')
Sort functions verified successfully.

All demonstrations and verifications completed successfully.

```

## Summary: Online Food Delivery Platform System

This solution implements a basic yet efficient online food delivery platform system in Python. It includes an `Order` class to manage order details and provides key functionalities for searching and sorting order records using appropriate algorithms.

### Key Components and Algorithm Choices:

#### 1. `Order` Class Definition:

- **Purpose:** Encapsulates order attributes such as `order_id`, `restaurant_name`, `delivery_time`, `price`, and `order_status`.
- **Benefit:** Provides a clear and structured way to represent individual order records, making the data model intuitive and easy to work with.

#### 2. Search Function (`search_by_order_id`):

- **Algorithm:** Hash Map (Python Dictionary).
- **Time Complexity:** Average  $O(1)$ .
- **Benefit:** Order IDs are unique identifiers. Using a hash map ensures near-instantaneous retrieval of an order by its ID. This is crucial for real-time operations like customer service inquiries, order tracking, and internal order management, where quick access to specific order details is paramount.

#### 3. Sort Functions (`sort_by_delivery_time` and `sort_by_price`):

- **Algorithm:** Timsort (Python's built-in `sorted()` function).
- **Time Complexity:**  $O(N \log N)$ .
- **Benefit:** Timsort is a highly optimized hybrid sorting algorithm that performs efficiently across various data distributions. Sorting by `delivery_time` is vital for logistics, enabling efficient routing and dispatching of delivery personnel. Sorting by



`price` allows for customer-friendly display options, filtering, and internal analysis of order values. The  $O(N \log N)$  complexity ensures that even with a growing number of orders, sorting operations remain performant and scalable.

## Overall Benefits:

The chosen algorithms provide a strong foundation for an efficient online food delivery platform:

- **High Performance:**  $O(1)$  lookups for order IDs and  $O(N \log N)$  sorting ensure that common operations are fast, even with large datasets.
- **Scalability:** The algorithmic complexities chosen scale well with an increasing number of orders, which is critical for a growing platform.
- **Maintainability:** Leveraging Python's built-in `sorted()` function simplifies the code and reduces potential for errors, while providing battle-tested performance.
- **User Experience:** Fast search and well-organized sort results directly contribute to a smoother experience for both customers tracking their orders and platform administrators managing operations.

## Final Task

### Subtask:

Summarize the Online Food Delivery Platform solution, highlighting the chosen algorithms and their benefits for managing order records efficiently.

## Summary:

### Data Analysis Key Findings

- **Algorithm Selection for Efficiency:**
  - **Search by Order ID:** A Hash Map (Python Dictionary) was chosen, providing an average time complexity of  $O(1)$ . This ensures nearly instantaneous retrieval of order details, critical for operations requiring quick access.
  - **Sorting (by Delivery Time and Price):** Timsort, implemented by Python's built-in `sorted()` function, was selected. It offers an efficient average and worst-case time complexity of  $O(N \log N)$ , suitable for diverse data and ensuring scalability for organizing orders.
- **Structured Order Data:** An `Order` class was defined to encapsulate order attributes such as ID, restaurant name, delivery time, price, and status, providing a clear and maintainable data model.
- **Successful Implementation and Verification:**
  - The `search_by_order_id` function was successfully implemented and verified to correctly find existing orders and return `None` for non-existent ones, leveraging the  $O(1)$  average time complexity of hash maps.
  - The `sort_by_delivery_time` and `sort_by_price` functions were successfully implemented and verified, demonstrating accurate sorting based on the respective attributes with  $O(N \log N)$  performance.
- **Comprehensive Demonstration:** A sample list of `Order` objects was created, and all implemented search and sort functions were thoroughly demonstrated and verified for correctness, confirming the system's functionality.

### Insights or Next Steps

- The current system provides a robust and efficient core for order management through judicious algorithm selection. To enhance real-world applicability, consider implementing data persistence (e.g., using a database) to store and retrieve order records permanently.
- Further development could include features like order updates, cancellation logic, and more complex sorting criteria (e.g., sorting by multiple fields or custom rules), along with user authentication and role-based access control for different platform users.

